

CLEAN ARCHITECTURE

프로그래밍 패러다임

2부: 벽돌부터 시작하기

3장 ~ 6장

2026.04.08

“

좋은 아키텍처는
'할 수 있는 것'을 늘리는 게 아니라,
'하면 안 되는 것'을 명확히 하는 것

— 3장, 패러다임 개요

구조적 프로그래밍: **쪼개라**

분해해야 테스트할 수 있다

BEFORE: 분해 전

```
OrderService.java

public void processOrder() {
    // 검증 로직 50줄
    // 할인 계산 30줄
    // DB 저장 20줄
    // 알림 발송 15줄
}
```

AFTER: 분해 후

```
OrderService.java

public void processOrder() {
    validate(order);
    Price p = calculateDiscount(order);
    save(order, p);
    notify(order);
}
```

의존성 역전이 **없는** 세계

제어 흐름 = 의존성 방향

OrderService



MySQLRepository



MySQL

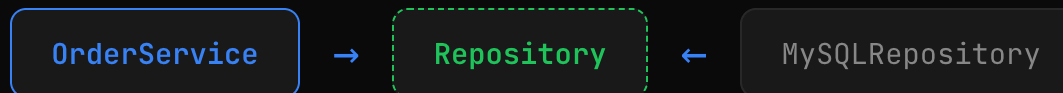
OrderService.java

```
// 직접 의존 - DB가 바뀌면 서비스도 수정해야 함
class OrderService {
    private MySQLRepository repo
        = new MySQLRepository();
}
```

상위 정책(비즈니스 로직)이 하위 세부사항(DB)에 종속

의존성 역전이 있는 세계

의존성 방향 ≠ 제어 흐름



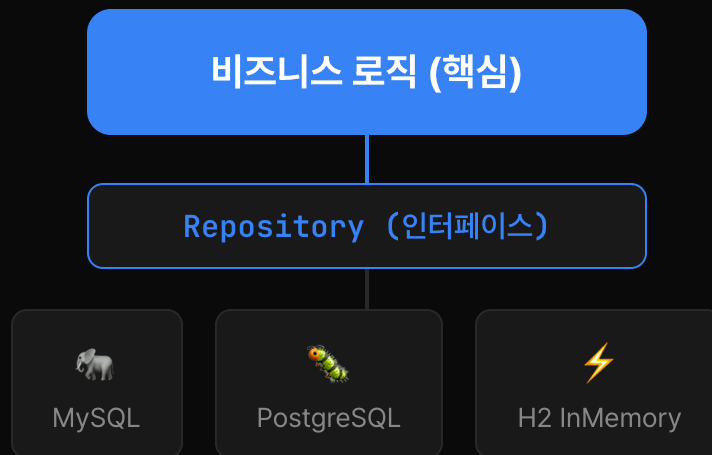
OrderService.java

```
// 인터페이스에만 의존 - DB가 바뀌어도 무관
class OrderService {
    private Repository repo;
    // 인터페이스에만 의존!
}
```

DB를 바뀌도 OrderService는 모른다

플러그인 아키텍처

이것이 클린 아키텍처 전체의 설계 원리



구현체를 플러그인처럼 교체해도 핵심 로직은 영향받지 않는다

불변 영역과 가변 영역의 격리

계산은 순수하게, I/O만 가변으로

불변 (순수 계산)

```
class PriceCalculator {  
    int calculateDiscountedPrice(  
        int price, int percent) {  
        return price  
            * (100 - percent) / 100;  
    }  
}
```

- ✓ DB 없이 단독 테스트 가능
- ✓ 다른 곳에서 재사용 가능
- ✓ 동시성 문제 없음

가변 (I/O)

```
class OrderService {  
    void applyDiscount(  
        Long id, int percent) {  
        Order o = repo.findById(id);  
        int p = calc.calculate(  
            o.getPrice(), percent);  
        o.setPrice(p);  
        repo.save(o);  
    }  
}
```

- ✗ DB 조회/저장만 담당
- ✗ 가변 범위를 최소한으로
- ✗ 여기만 동시성 주의

세 패러다임이 아키텍처에 준 것



구조적

빼앗은 것

무질서한 제어 흐름

아키텍처에 준 것

테스트 가능한 분해



객체지향

빼앗은 것

무분별한 간접 호출

아키텍처에 준 것

의존성 방향 제어



함수형

빼앗은 것

변수 재할당 (상태 변경)

아키텍처에 준 것

동시성 안전

같이 생각해볼 질문



구현체가 하나뿐인 Repository에도
인터페이스가 필요한가?

DIP는 강력하지만, 이건 오버엔지니어링 아닌가?
아니면 테스트를 위해서라도 필요한가?

패러다임은 할 수 있는 것을 늘리지 않는다

하면 안 되는 것을 정해준다.

그 제약이 좋은 아키텍처의 시작이다.

클린 아키텍처 2부: 3장 ~ 6장